3/ Développer l'application (Front-end & Back-end)

1. Développement du Front-end :

a. Création des interfaces avec HTML, CSS, JS

Le front-end sera basé sur les pages principales suivantes :

- **Page d'accueil** : Affiche les produits en vedette et les catégories de produits.
- **Page de produits** : Liste des produits disponibles avec la possibilité de filtrer par catégorie ou prix.
- **Page de détails produit** : Informations détaillées sur chaque produit.
- **Panier** : Vue des produits ajoutés au panier et possibilité de passer à la caisse.
- **Page de paiement** : Formulaire pour l'utilisateur afin de finaliser l'achat.

Exemple d'une page HTML pour la page d'accueil :

```
<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <meta name="viewport" content="width=device-width, initial-scale=1.0">

   <title>FashionStore</title>

   <link rel="stylesheet" href="styles.css">
```

```
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0-alpha1/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
  <header>
    <nav class="navbar navbar-expand-lg navbar-light bg-light">
      <a class="navbar-brand" href="#">FashionStore</a>
      <ul class="navbar-nav ms-auto">
        <li class="nav-item"><a class="nav-link" href="#">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Products</a></li>
        <li class="nav-item"><a class="nav-link" href="#">Cart</a></li>
      </ul>
    </nav>
  </header>

  <main class="container mt-4">
    <div class="row">
      <div class="col-md-4">
        <div class="card">
          <img src="product1.jpg" class="card-img-top" alt="Product Image">
          <div class="card-body">
```

```
                <h5 class="card-title">Stylish Jacket</h5>
                <p class="card-text">$49.99</p>
                <a href="#" class="btn btn-primary">Add to Cart</a>
            </div>
        </div>
    </div>
    <!-- Other product cards -->
  </div>
</main>

<footer class="bg-light text-center py-3">
  <p>&copy; 2025 FashionStore</p>
</footer>

<script src="scripts.js"></script>
</body>
</html>
```

b. Utilisation de Bootstrap, React ou autres Frameworks

- **Bootstrap** est utilisé ici pour simplifier la mise en page et rendre le site réactif.
- **React** pourrait être utilisé pour créer une interface utilisateur dynamique, particulièrement pour la gestion du panier d'achats, la mise à jour des produits et la gestion de l'authentification des utilisateurs.

c. Test et validation des interfaces

- Tester le site sur différents navigateurs pour s'assurer qu'il est responsive et accessible.
- Valider les formulaires d'inscription et de paiement.

2. Développement du Back-end :

a. Création des API et gestion des bases de données avec Laravel

- **Base de données** : Une base de données MySQL est utilisée pour stocker les utilisateurs, produits, commandes, et paniers.
- **API REST** : Création d'API pour gérer les produits, le panier, et les commandes.
    - GET /api/products : Récupère tous les produits.
    - POST /api/cart : Ajoute un produit au panier.
    - POST /api/checkout : Finalise la commande.

Exemple d'une route API Laravel pour récupérer les produits :

```
use App\Models\Product;
use Illuminate\Support\Facades\Route;

Route::get('/products', function () {
   return Product::all();
});
```

b. Codage des fonctionnalités et gestion des données

- **Gestion des produits** : CRUD pour ajouter, modifier, et supprimer des produits.
- **Gestion du panier** : Ajout des produits au panier, modification de la quantité, et suppression des produits.

- **Gestion des commandes** : Récupération des informations de paiement et gestion des commandes.

c. Optimisation des performances et tests de sécurité

- Utiliser la **mise en cache** pour les produits populaires afin d'améliorer la vitesse de chargement.
- **Validation des entrées** avec Laravel pour éviter les injections SQL et les failles de sécurité.
- Utiliser **HTTPS** pour sécuriser les échanges entre le client et le serveur.

3. Sécurisation de l'application :

a. Gestion des rôles et accès des utilisateurs

- **Authentification** avec Laravel Passport ou JWT pour gérer les utilisateurs (administrateurs, clients).
- **Middleware** pour contrôler l'accès aux ressources en fonction des rôles.
    - admin peut ajouter/modifier/supprimer des produits.
    - client peut seulement passer des commandes et consulter les produits.

Exemple de middleware pour gérer l'accès :

```
public function handle($request, Closure $next)
{
   if (auth()->user()->role != 'admin') {
      return response()->json(['error' => 'Unauthorized'], 403);
   }
```

```
    return $next($request);
}
```

b. Sécurisation des données stockées et des échanges

- **Cryptage des mots de passe** avec bcrypt() lors de l'enregistrement des utilisateurs.
- **SSL/TLS** pour sécuriser les échanges entre le front-end et le back-end.
- **Validation des entrées** pour empêcher les attaques par injection SQL.

4. Exemple de fonctionnement complet :

1. L'utilisateur se rend sur la **page d'accueil**, consulte les produits et les ajoute à son **panier**.
2. Lorsqu'il est prêt à acheter, il se connecte ou s'inscrit, puis passe à la **page de paiement**.
3. Une fois la commande validée, les données sont envoyées à l'API Laravel pour la création d'une commande.
4. Le système effectue un paiement simulé (via une API de paiement tierce) et confirme la commande.